

Node.js Interview Questions And Answers For Freshers

Q1. What is Node.js?

Ans: Node.js is a cross-platform, open-source runtime environment that can run web applications. It's commonly used as a server-side language for application programming, and its asynchronous and event-driven structure mainly assists with data-intensive applications. Moreover, Node.js enables developers to run JavaScript code on a machine outside a browser.

Q2. What is the purpose of Node.js?

Ans: Node.js is built as a single-threaded architecture, running only on event-driven, non-blocking servers. It is used for building:

- Network applications
- Real-time web applications
- Distributed systems
- General-use software

Q3. What are the benefits of using Node.js?

Ans: Here are the critical benefits of Node.js:

1. **Full-Stack JavaScript Solution:** Node.js allows for creating a full-stack JavaScript application as it supports server-side and client-side coding. Companies can save time and money as they do not have to hire different back-end and front-end developers.
2. **Significant Customization Ability and In-built APIs:** Node.js is very flexible, allowing for heavy customization to suit particular needs. It supports easy transfer of data between the server and client using JSON and arrives with inbuilt APIs for serving development use, such as HTTP, DNS, and TCP.
3. **Excellence in Performance:** As the code is converted directly into machine code with the aid of Google's V8 JavaScript engine, Node.js delivers excellent performance. This is one reason it is favoured in back-end development, especially with big data set operations.
4. **Scalability:** Inherent scalability is another feature of Node.js. It runs multiple nodes and lets their interaction take place. Therefore, its workload balancing is a good reason why it needs to be chosen for back-end development in a cross-platform environment.
5. **Caching Capabilities:** Node.js has a cache module that enhances performance by making pages load faster. Since data is read from memory, request processing is faster, and responses are error-free and quick for users to access web applications.

Q4. Why is Node.js Single-threaded?

Ans: Node.js uses a single thread to efficiently handle asynchronous processing. The operation of asynchronous tasks in a single thread achieves better performance and scalability than typical thread-based approaches under usual web loads.

Q5. If Node.js is single-threaded, then how does it handle concurrency?

Ans: Node.js is not based on the Multi-Threaded Request/Response Stateless Model. It relies on the Single-Threaded Event Loop Model, inspired by JavaScript's architecture of the event-driven system with callbacks, allowing Node.js to manage numerous client requests that can come in concurrently. The core of the above processing model is an event loop

Q6. Define callback in Node.js?

Ans: A callback is a function that gets executed after completing a task. This way, other code runs parallel, and the program won't block. Being an asynchronous platform, Node.js relies heavily on callbacks to keep the operation efficient and all its library APIs were designed to support them.

Q7. What is NPM?

Ans: NPM, short for Node Package Manager, manages all packages and modules for Node.js. The two primary functions include:

- **Online Package Storage:** NPM has a centralised repository of Node.js packages and modules that can be browsed at search.nodejs.org.
- **Command-line utility:** It will enable a command-line interface to download Node.js packages, manage dependencies, and deal with multiple versions of Node.js.

Q8. What is I/O?

Ans: I/O, or Input/Output, refers to how a program switches data to and from the outside world, from external disk storage, network connections, or user interfaces. Since many I/O operations typically involve physical hardware and other system resources outside of memory and the CPU, they tend to be slower than memory-bound or CPU-bound operations. Let us look at some I/O types.

- **Disk I/O:** This includes reading from or writing to a disk-hard drive or solid-state drives-as well as saving files, accessing databases, or loading data from storage. Not surprisingly, disk I/O is often the bottleneck in many systems because accessing data from disk is much slower than accessing it from memory.
- **Network I/O:** This refers to sending or receiving data over a network. Examples are making HTTP requests, accessing an API, or the operations of remote servers. Network I/O can be

impacted significantly by factors such as network latency, bandwidth, and server response times, which makes network I/O slower than in-memory operations. This is user interaction through keyboards, mice, or touchscreens. For instance, accept user input, show results, and produce reports.

- **Database I/O:** This encompasses reading from or writing data to a database. Databases are located on a separate server or network; operations, such as querying, inserting, or updating records involve I/O operations that can be time-consuming.

Q9. What is the difference between Angular and Node.js?

Ans: Here are the significant differences between Angular and Node.js:

Angular	Node.js
An open-source framework for developing web applications	A cross-platform runtime environment for application development
Developed using TypeScript	Built using C, C++, and JavaScript
Primarily used to create single-page applications for the client-side.	Perfect for creating fast, scalable server-side networking applications
Ideal for developing interactive and engaging web applications	Ideal for creating lightweight applications.
Perfect for building real-time applications.	Perfect for scenarios that demand scalability and high speed.

Q10. How many types of API functions are there in Node.js?

Ans: Node.js has two flavours of API functions:

- **Asynchronous, Non-blocking API Functions:** They operate in isolation, meaning that when Node.js requests data from an API, it does not wait for the response before attending to another activity. Instead, it takes up other operations, and once the data is received, this becomes an event notification, which will trigger a response regarding the previous request. It simply translates to allowing tasks to run concurrently while handling requests. These are various examples: online forums and email notifications.
- **Synchronous, Blocking API Functions:** Here, the system waits for a response again. The calling system in this scenario expects instant data after the call. Such APIs are beneficial in scenarios that need high availability and low latency. In short, the application sends a request to the server and waits for a response before proceeding. Some of the examples are instant messaging, video conferencing, etc.

Q11. Explain Node.js web application architecture.

Ans: NPM, short for Node Package Manager, manages all packages and modules for Node.js. The two primary functions include:

1. **Online Package Storage:** NPM has a centralised repository of Node.js packages and modules that can be browsed at search.nodejs.org.
2. **Command-line utility:** It will enable a command-line interface to download Node.js packages, manage dependencies, and deal with multiple versions of Node.js.

Q12. Why is Node.js favoured over other backend technologies like Java and PHP?

Ans: Node.js is preferred over other backend technologies such as Java and PHP for the following reasons:

- **Speed:** Node.js performs well with its non-blocking, event-driven architecture.
- **NPM:** Node Package Manager supplies more than 50,000 packages; this makes available a rich set of tools and libraries to developers for efficient development.
- **Real-time and Data-intensive Web Applications:** Node.js is perfect for building real-time, data-intensive web applications because it does not wait for API responses and can process multiple requests simultaneously.
- **Code Synchronisation:** With Node.js, developers can use the same codebase for server and client-side programming, improving synchronisation.
- **Easy to Use:** Because Node.js uses JavaScript, web developers who are familiar with JavaScript can easily bring it into their projects without learning a new language.

Q13. What is the difference between front-end and back-end development?

Ans: Here are the significant difference between front-end and back-end development:

Front-end Development	Back-end Development
Front-end development refers to the client-side of an application.	Back-end development pertains to the server-side of an application.
It is the portion of web applications that users can view and interact with.	It includes all the backend components that are not visible or interactive to the user.
It encompasses all the elements that contribute to the visual aspects of a web application.	It involves a web server that interacts with the database to fulfill the users' requests.

Examples of front-end development technologies include CSS, HTML, JavaScript, AngularJS, Bootstrap, jQuery, and React.js. These technologies are used to create and design the visual elements and user interactions of a web application.	.Some examples of back-end development technologies include Java, PHP, Python, C++, Node.js, and others. These technologies are used to handle the server-side logic, database interactions, and functionality that support the front-end of a web application.
It is used for SEO	It is used for backup
Examples of front-end frameworks include AngularJS, React.js, jQuery, Sass, and others.	Examples of back-end frameworks include Express, Django, Rails, Laravel, Spring, and others.

Node.js Interview Questions And Answers For Intermediate-Level

Here are some Node.js interview questions and answers for the intermediate-level candidate that you should know:

Q1. How can you manage the packages in the Node.js project?

Ans: In a Node.js project, one can use different package installers and their respective configuration files to manage packages. However, most projects utilise NPM or Yarn, both of which feature extensive libraries in JavaScript and allow environment-specific configurations. The package.json and the package-lock.json file help control the version of libraries installed within a project.

Q2. What is REPL in the context of Node.js?

Ans: In the Node.js framework, REPL is an abbreviation for Read, Eval, Print, and Loop. It's an environment where one can interact by entering commands within it, similar to a console or Linux terminal and which prints out the system's responses to commands. The tasks performed by REPL are as follows:

- **Read:** The input from the user is read and parsed into a JavaScript data structure and then stored in memory.
- **Eval:** The data structure is evaluated.
- **Print:** The outcome of the evaluation is printed.
- **Loop:** Continues on, running commands till the user presses CTRL+C twice.

Q3. What is event-driven programming?

Ans: Event-driven programming is a paradigm in which some activity, known as an event, triggers the execution of specific functions or actions. The event can be anything such as a key press or

mouse click. At any time when the event happens, a previously registered callback function is executed.

Q4. What are the challenges faced in Node.js?

Ans: These are the major challenges faced in Node.js:

- **Vast stack:** Node.js does not offer any particular custom to develop an application. The frameworks used by Node.js are not opinionated and do not provide any specific guidelines for writing the codes. It is one of the reasons why each application needs a different and individual approach and experienced programmers for developing and maintaining the codes.
- **Technical errors:** Several problems in Node.js occur because of a lack of practice. The open-source libraries and frameworks are highly active and provide numerous ways to build applications. However, having many options can be a major problem for developers with zero experience, leading to application structure incompatibility.
- **Scalability problems:** Node.js is a single-threaded approach that makes the scaling a bit confusing. However, you might need to divide the complex applications with CPU/MEM-heavy computations into smaller microservices, which manage different operations. Hence, when you create a scalable Node.js application, you must keep it small.
- **Poor documentation:** Documentation is one of the essential parts of any IT project. It gives an insight into how applications work and informs the developers about the components, how they are relatable, and what their significant function is. Poor documentation will extend the developing time and make the entire process difficult to understand. It can create multiple problems when it comes to the implementation of new features and performance.

Q5. What are the key features of Node.js?

Ans: Here are some key features of Node.js:

- Node.js is asynchronous and event-driven, meaning that if Node receives a request from some Input/Output operation, it will execute and carry out that operation in the background along with accepting other requests. Hence, it will not wait for the response to the previous requests.
- It is fast in executing code, as Node.js uses the V8 JavaScript Runtime engine used by Google Chrome. It has a covering over the JavaScript engine that makes the runtime engine faster; hence, the processing of requests in Node.js also becomes faster.
- Node.js operates a single-thread model for event looping. The response from these events might or might not reach the server instantly. However, it does not stop other operations from happening in the background and makes Node.js highly scalable. Traditional servers create limited threads to manage the requests, whereas Node.js creates a single thread which offers service to a larger number of requests.
- Node.js library uses JavaScript, which is also one of the essential components of Node.js for developers. Most developers know Javascript, its features, and its usage. Because of this, development in Node.js becomes easier for a developer who knows JavaScript.

Q6. What is callback hell in Node.js?

Ans: Callback hell, or the Pyramid of Doom, is a situation in which callbacks are deep-level nested; the code becomes unruly. Every such callback depends on the output of the previous one, resulting in a confusing structure in the program that makes it harder to read and understand and reduces its maintainability. The problem arises from the execution flow of the asynchronous operations themselves.

Q7. What is the meaning of a test pyramid?

Ans: A test pyramid is an image showing the ideal ratio between unit tests, integration tests, and end-to-end tests required in testing a project for effective development.

Q8. What do you understand about middleware in Node.js?

Ans: Middleware is a function that takes the Request and Response object as arguments. In the request-response cycle of an application, these functions have access to multiple request and response objects, as well as the next function in the cycle. Some of the most common things middleware functions do are:

- Execute some custom code
- Passing control to the next middleware in the stack:
- Changing or updating the request and response objects
- Completing the request-response process

Q9. How many types of streams are available in Node.js?

Ans: There are the following types of streams available with Node.js:

- **Readable:** This stream is meant to read data.
- **Writable:** This stream is used to write data.
- **Duplex:** This stream supports reading and writing operations simultaneously.
- **Transform:** This stream processes data; it changes the output according to the input.

Q10. What is the Reactor Pattern in Node.js?

Ans: The Reactor Pattern is a concept that focuses on non-blocking I/O operations and is designed to prevent blocking during input/output tasks. It will allow a developer to have a handler for each I/O operation. When I/O requests are made, they are sent to the demultiplexer, essentially a notification interface that handles concurrency through non-blocking I/O mode, and collects every request as an event and places them in a queue to be processed.

Q11. What are LTS releases of Node.js?

Ans: LTS, or Long Term Support, is a version of Node.js with critical bug fixes, security updates, and performance improvements. They are supported for at least 18 months and put in stability for security. LTS version changes will normally focus on bug fixes, npm updates, security patches, performance enhancements, and documentation improvements.

Node.js Interview Questions And Answers For Experienced Professionals

Q1. What are the different modules used in Node.js?

Ans: Modules in Node.js are like JavaScript libraries, a collection of functions. To include a module in a Node.js application, developers should put the module's name in parentheses.

There exist many modules in Node.js and they provide most of the functionalities for a web application, some of which include :

Core Modules	Description
HTTP:	The HTTP module provides classes, events, and methods to create an HTTP server in Node.js.
Url:	The URL module offers methods for working with and determining URLs
Util:	The util module supplies utility functions essential for application development and useful for developers
Fs:	The FS module provides methods, classes, and events to handle file I/O operations.
Query string:	The query string module offers methods for working with query strings.
Stream:	The stream module provides methods to manage and process streaming data.
Zlib:	The zlib module provides methods to compress and decompress files, optimising their size for use in applications.

Q2. How is clustering used to enhance the performance of Node.js?

Ans: Clustering is used to overcome the limitation of a single processor. When clusters are used in a Node.js application, several Node.js processes are created. Yet there is also a parent process called the cluster manager. It is responsible for monitoring the health of the individual application instances, guaranteeing that everything is working fine.

Q3. Is cryptography supported in Node.js?

Ans: Yes, Node.js supports cryptography through a module known as Crypto. It provides various cryptographic functionalities like deciphering, ciphering, a set of wrappers for opening SSL's hash HMAC, verifying functions, etc.

Q4. Define why Express 'app' and 'server' must be kept separate.

Ans: Keeping the Express app and server separate is to hold down coupling and better organise the internal application logic. Here are a few more points about this separation:

- To test the API process, one can forego network calls.
- It enables faster test execution.
- It comes with a better separation of concerns and results in cleaner code.
- It simplifies deploying the same API with distinct, flexible network configurations.

Q5. What is libuv in Node.js?

Ans: The libuv is a multi-platform support library, and Node.js enunciates asynchronous I/O capabilities. Initially developed for Node.js, it has since been adopted by other popular systems such as Julia, Luvit, and pyuv. Some of its key features include:

- Asynchronous file and file system operations.
- Handling child processes
- File system events
- Asynchronous TCP and UDP sockets
- A fully functional event loop system

Q6. Explain an Event Emitter in Node.js.

Ans: The EventEmitter is a class in Node.js representing objects that can emit events. These objects include the `eventEmitter.on()` function, which allows multiple functions to be associated with specific named events emitted by the object. Whenever an event is emitted by an EventEmitter object, all functions hooked to that event are called synchronously.

Q7. What is the usage of buffer class in Node.js?

Ans: The Buffer class in Node.js is used for handling raw binary data in an array of integers. This operates outside of the V8 heap, using raw memory allocation. The Buffer class is immediately available and does not need a specific buffer module added. In layman's language, the Buffer class is needed because JavaScript natively does not support working with binary data.

Q8. Explain Punycode in Node.js.

Ans: Punycode is an encoding scheme representing a Unicode string of characters as a basic ASCII string. It is necessary to encode domain names because only ASCII characters are recognized by hostnames. Informally, Punycode is a character encoding, as defined by RFC 3492, designed primarily to be used in Internationalized Domain Names (IDNs).

Q9. Name some of the exit codes of Node.js.

Ans: Exit codes are a set of codes that signal completion or the result of a given procedure, including the global object in Node.js. Here are some of the commonly used exit codes in Node.js:

- Unused
- Fatal Error
- Uncaught Fatal Exception
- Internal Exception Handler Runtime Error
- Internal JavaScript Evaluation Failure

Q10. How Node.js handles the child threads?

Ans: Although Node.js runs as a single-threaded process and does not expose child thread management techniques, it still efficiently supports certain asynchronous I/O operations that execute in the background and do not run any JavaScript code or block the event loop.

If you are working on an application that requires child threading, then the ChildProcess module must be included within your development environment.

And that's the end of our Node.js interview questions and answers. Hope you have gained good insights from this blog that can be used to enhance knowledge. Now, let's practise these questions to bring out confidence in your job interview. Best of luck!